

CAP Theorem

ACID versus BASE consistency models

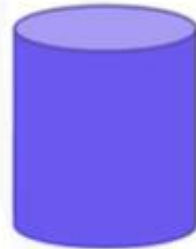


ACID database use cases

Financial institutions: Money transfers

Debit 100USD from xxx account and credit it to yyy account

ACID database



xxx 200USD

yyy 300USD

xxx 100USD

yyy 400USD





BASE Explained

1. Basically Available

The system guarantees availability—data is available most of the time, even in the face of partial system failures.

2. Soft State

The state of the system may change over time, even without input. This is due to eventual propagation of data across replicas.

3. Eventually Consistent

The system will become consistent over time, given that no new updates are made. It relaxes the consistency requirement in favor of availability and partition tolerance.



Use Case Example

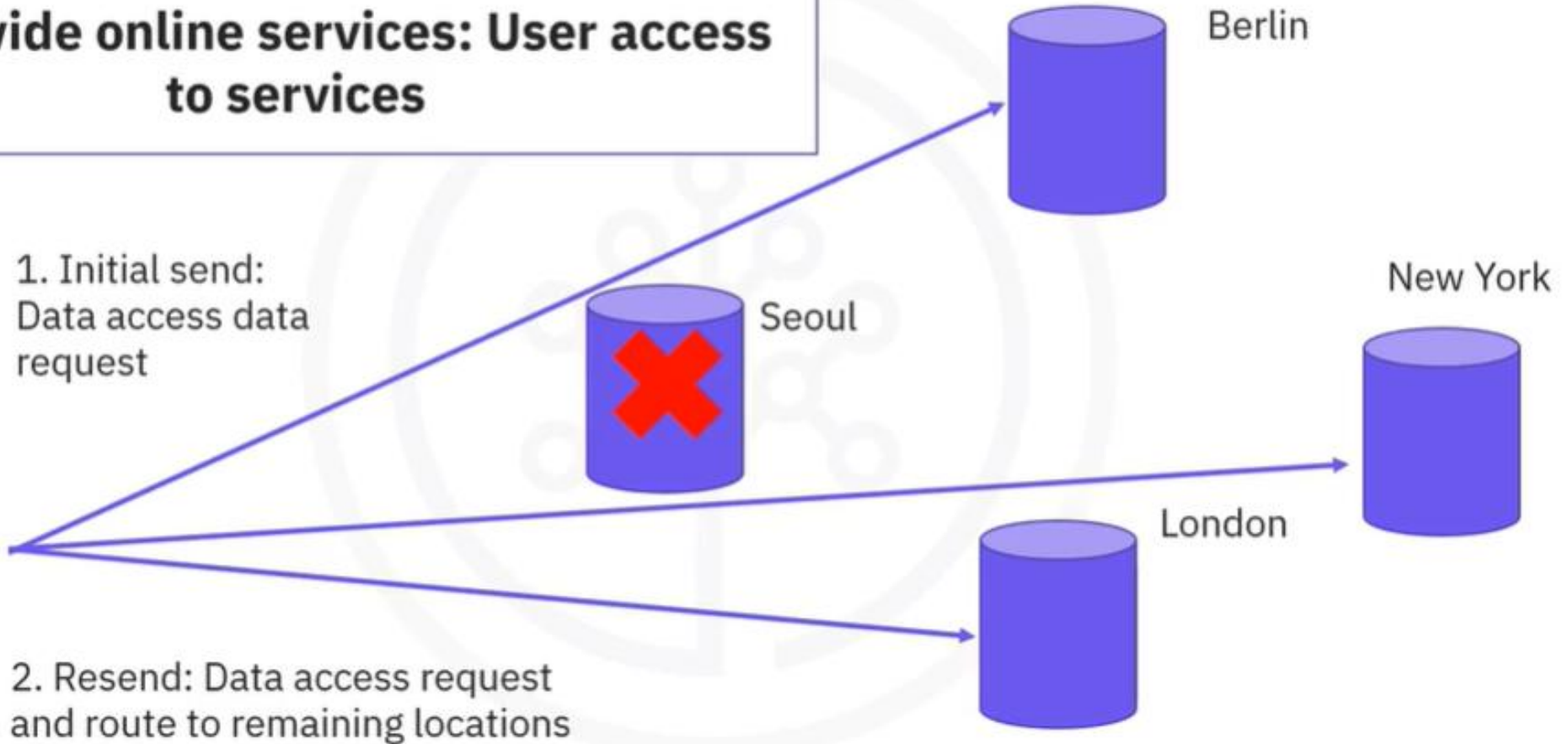
- In systems like **Amazon DynamoDB** or **Apache Cassandra**, data written to one node may not immediately be visible to all nodes (due to replication lag), but will eventually sync.
 - This is acceptable in **social media**, **logging**, **analytics**, and **content delivery** systems, where immediate consistency is less critical than performance and availability.
- 

NoSQL and BASE models

- NoSQL has few requirements for immediate consistency, data freshness, and accuracy
- NoSQL benefits: availability, scale, and resilience
- Used by:
 - Marketing and customer service companies
 - Social media apps
 - Worldwide available online services
- Favors availability over consistency of data
- NoSQL databases use BASE consistency model
- Fully available system

BASE database use cases

Worldwide online services: User access to services





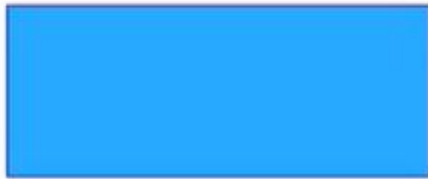
Skills
Network



Distributed Databases

Fragmentation

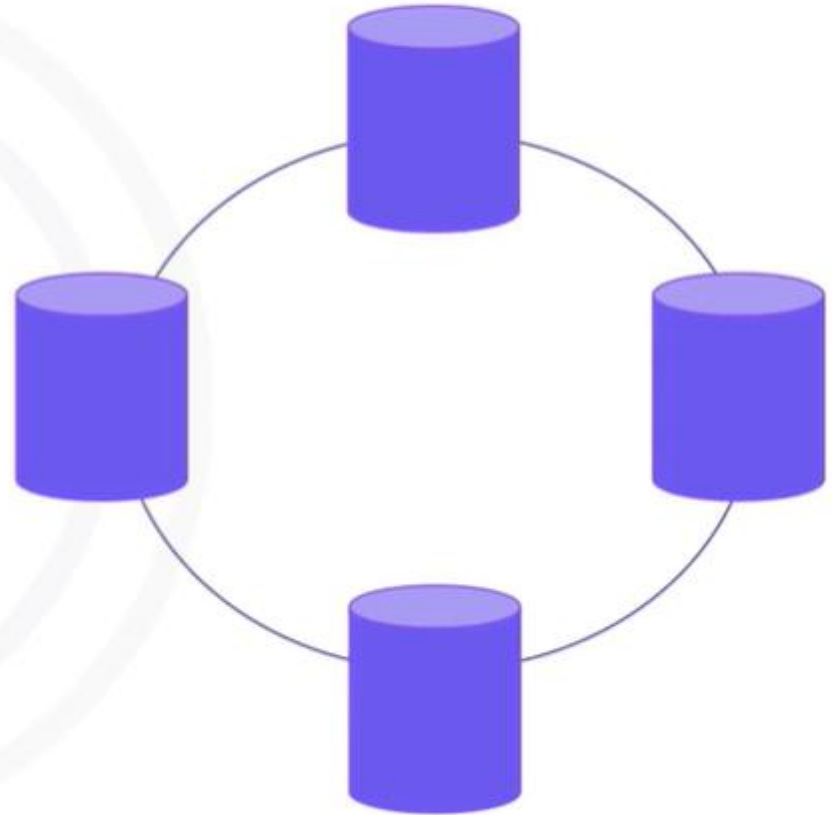
Input data



Fragmented input data

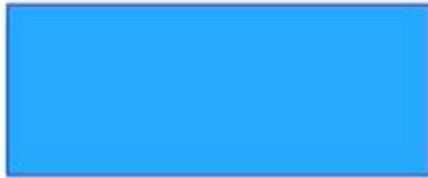


- Fragmenting your data (partitioning, sharding)

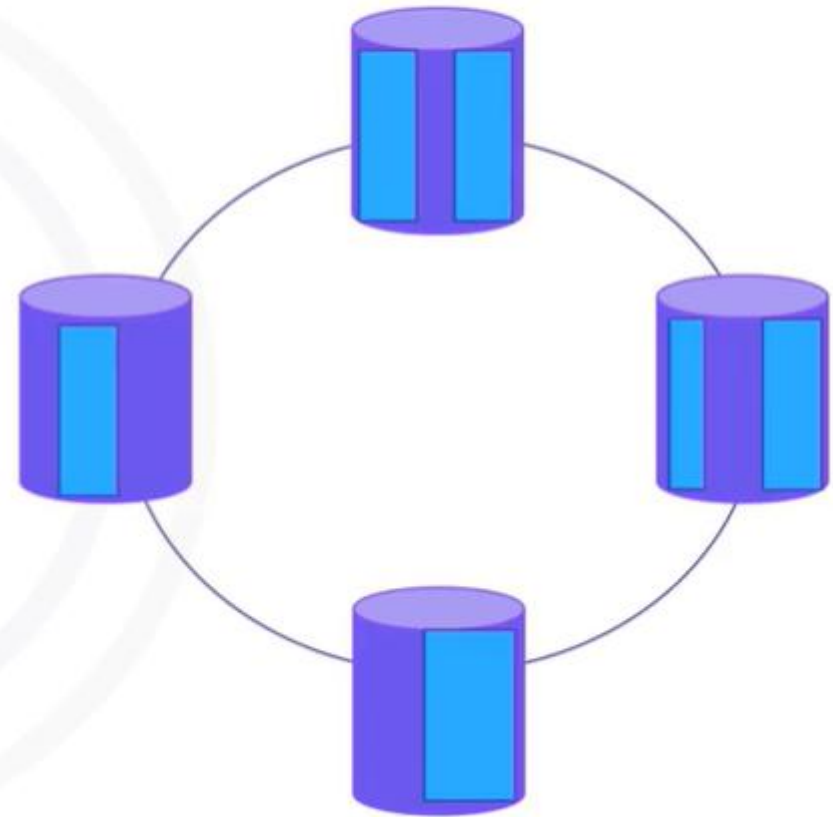


Fragmentation

Input data



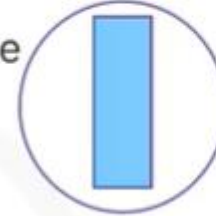
- Fragmenting your data (partitioning, sharding)
 - Grouping keys lexically
For example, all records starting with A or A-C)
 - Grouping records by key
For example, all records with key StoreId = 123)



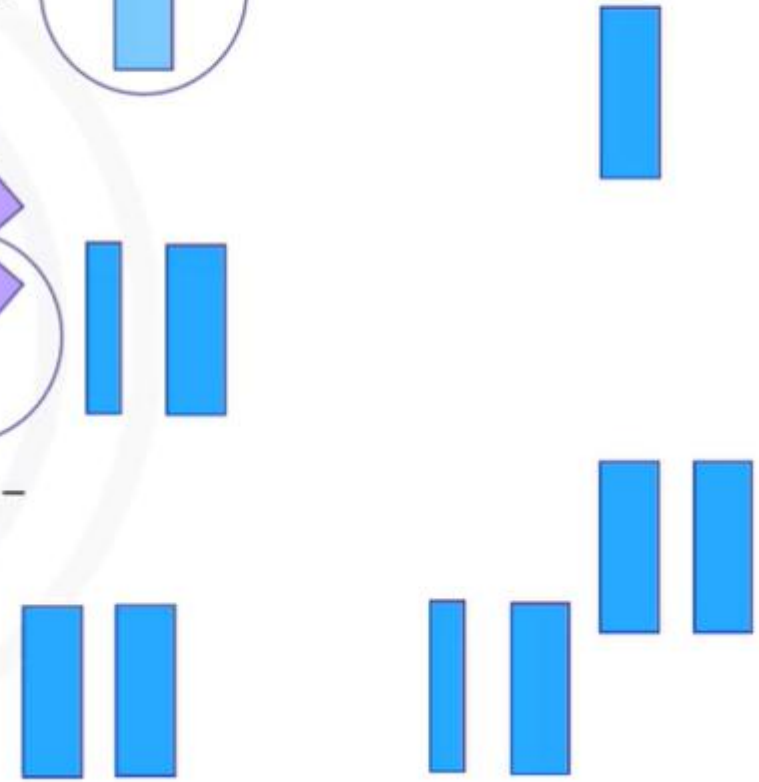
Replication

- Protection of data for node failures
- Replication: all data fragments are stored redundantly in two or more sites
- Increases the availability of the data
- Replicated data needs to be synchronized => could lead to inconsistencies

Data can be recovered from this node

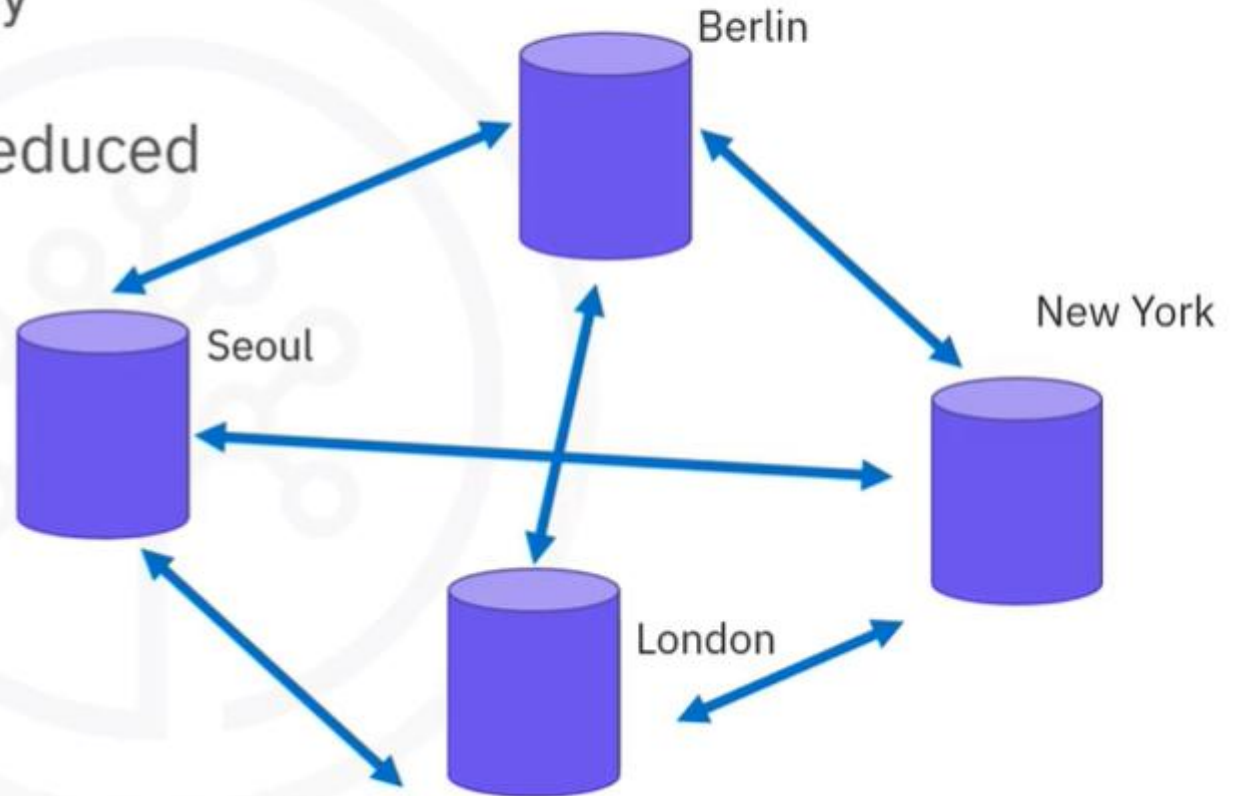


Node fails – data lost?



Advantages of distributed systems

- Reliability and availability
- Improved performance
- Query processing time reduced
- Ease of growth/scale
- Continuous availability



Distributed database challenges

- Distributed databases bring availability, fast scaling, and global reach capabilities
- Challenge: Concurrency control => consistency of data
 - WRITES/READS to a single node per fragment of data => data is synchronized in the background
 - WRITE operations go to all nodes holding a fragment of data, READS to a subset of nodes per Consistency
 - Developer-driven consistency of data
- No Transaction support (or very limited)

Distributed systems – BASE model

**Basically
Available**

Soft state

BASE

**Eventually
consistent**

CAP Theorem

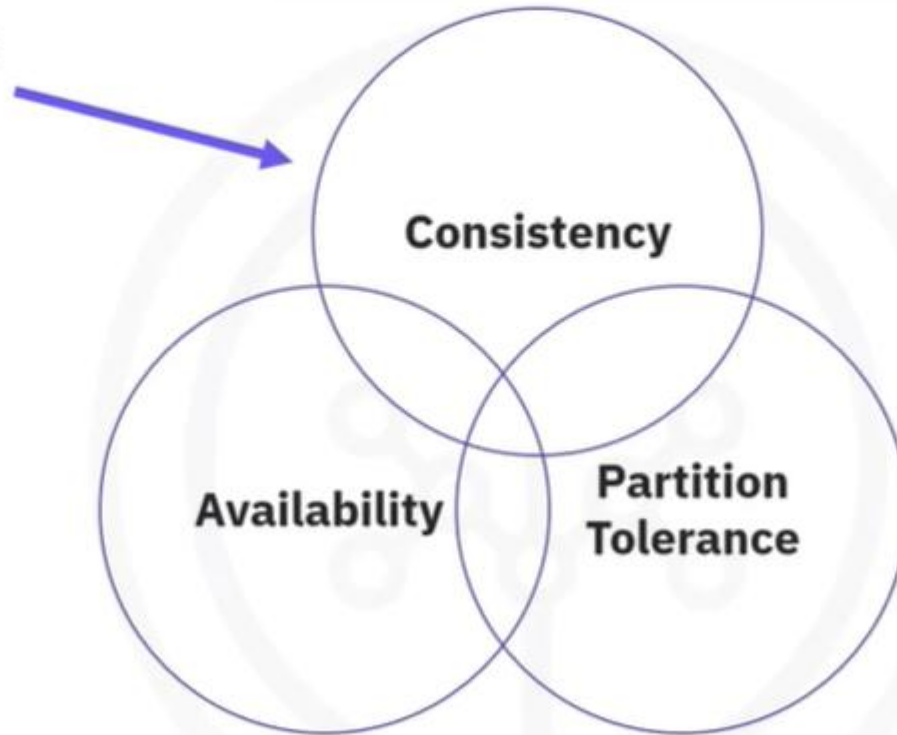
- The CAP (Consistency, Availability, Partition tolerance) theorem states that, for any distributed computer system it is impossible to simultaneously guarantee all three of the following characteristics:
 - Consistency
 - Availability
 - Partition tolerance

Property	Meaning
Consistency (C)	Every read receives the most recent write — all nodes return the same data.
Availability (A)	Every request receives a (non-error) response — even if some nodes fail.
Partition Tolerance (P)	The system continues to work even if communication between nodes is lost (i.e., network partition).



CAP theorem

All clients see the
same data at the
same time



CAP theorem

All clients see the same data at the same time



Consistency

Availability

Partition Tolerance

The system continues to operate even in the presence of node failures



CAP theorem

All clients see the same data at the same time



Consistency

The system continues to operate despite network failures

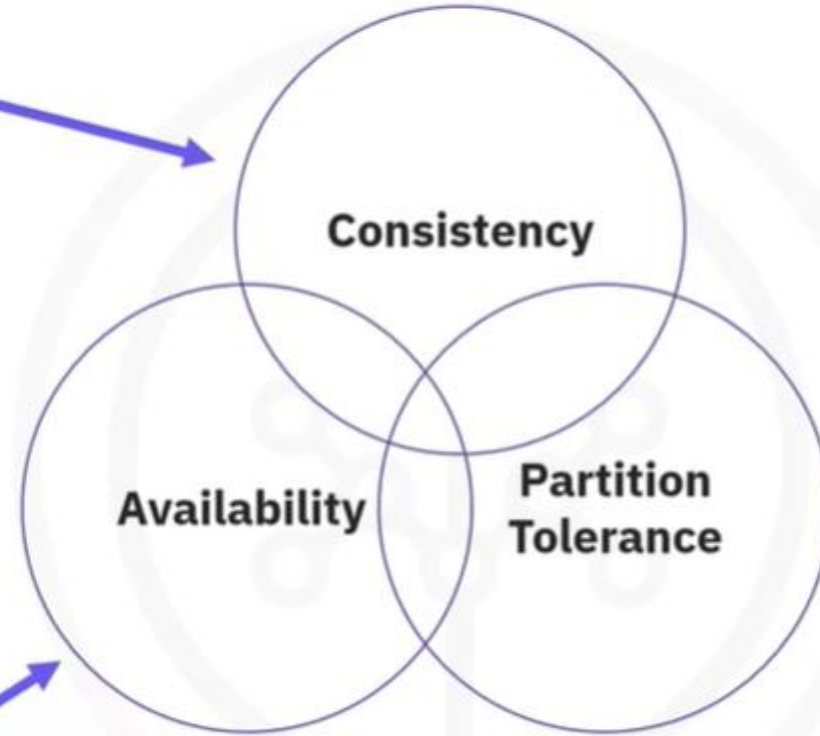


Partition Tolerance

The system continues to operate even in the presence of node failures



Availability

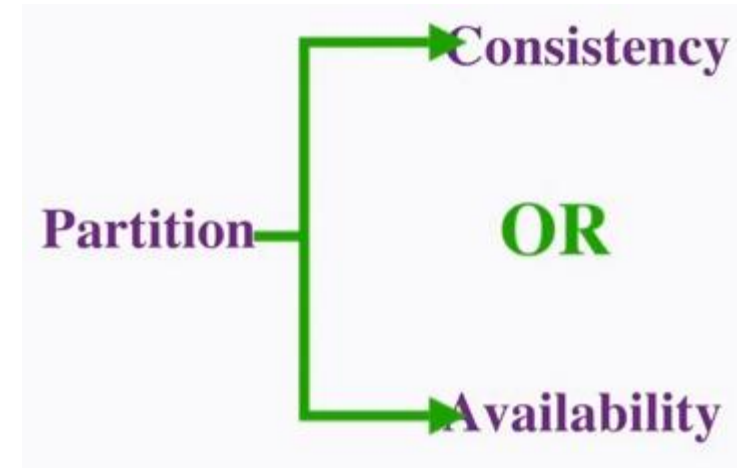


CAP Theorem

- According to the theorem, at same time, a distributed system cannot guarantee all three of these characteristics, utmost it may satisfy two out of three characteristics; hence, sometimes it also known as "two out of three" concept.
 - Consistency
 - Availability
 - Partition tolerance

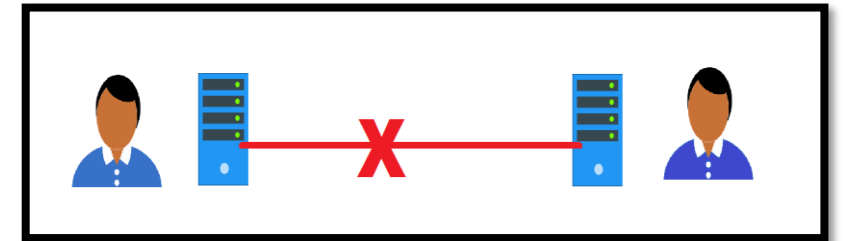
CAP Theorem

- It means that if a system is partitioned in a network environment, then we can achieve either consistency of the system or availability of the system



CAP Theorem

- Thus, CAP theorem states that for any distributed computer system it is impossible to have consistency, availability, and partition tolerance i.e. the three characteristics to be available at the same time.
- Generally, depending upon the need of an application, it is a choice between "two out of three".

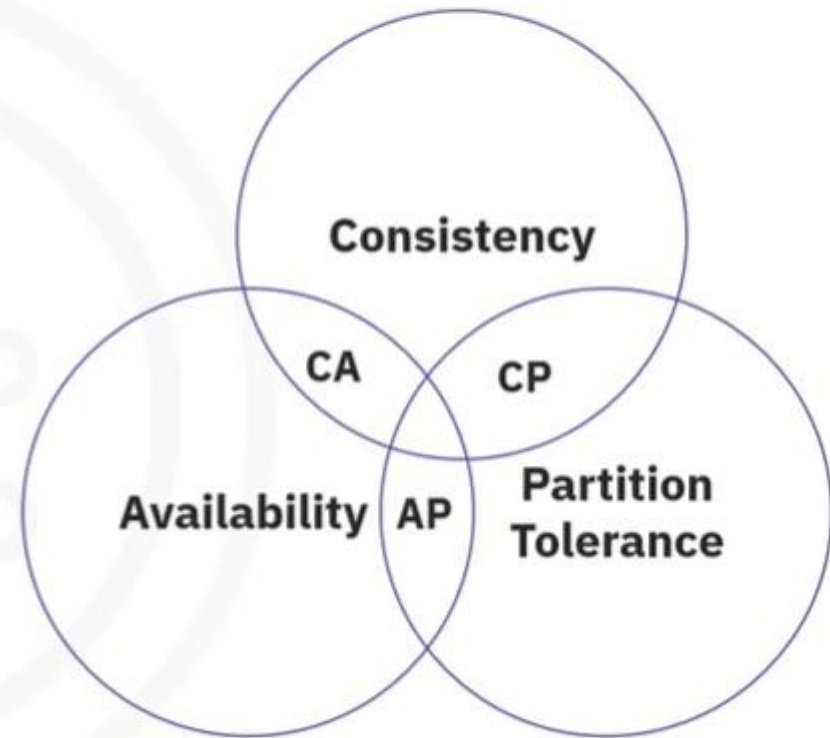


CAP Theorem

- NoSQL databases have loosened up the requirements on Consistency in order to achieve better Availability and Partitioning.
- This resulted in systems known as BASE (Basically Available, Soft-state, Eventually consistent).

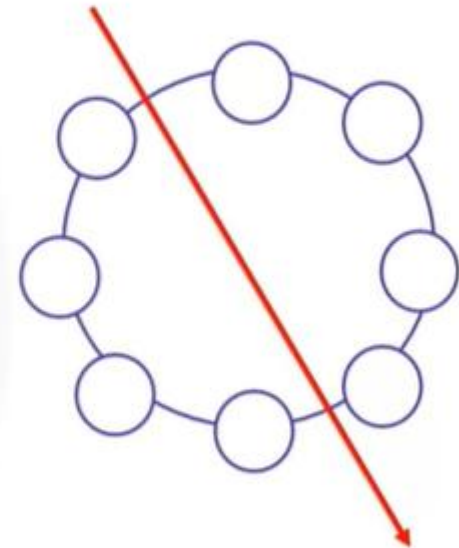
CAP theorem definition and history

- CAP Theorem (Brewer's theorem)
- Evolved by MIT professors Seth Gilbert and Nancy Lynch
- Consistency, Availability and Partition Tolerance (CAP)
- Only two of the three characteristics can be guaranteed in distributed systems



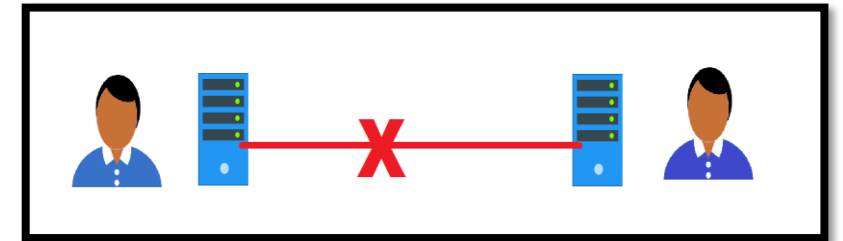
Partition tolerance

- A Partition is lost or temporarily delayed connection between nodes
 - Partition tolerance means that the cluster must work despite network issues
 - Distributed systems cannot avoid partitions and must be partition-tolerant
- Partition tolerance is a basic NoSQL feature



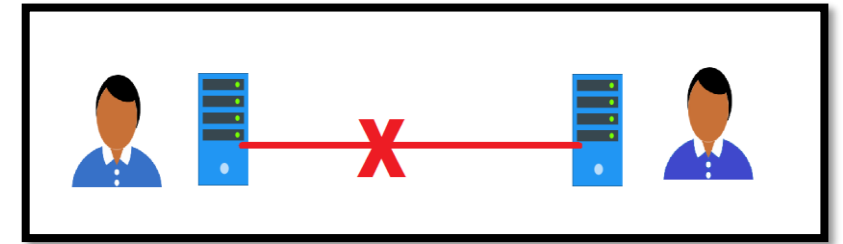
CAP Theorem

- Thus, if a network link is broken between two sites in a distributed environment, then any update operation on site X will not be reflected at site Y.
- Thus, the system will become inconsistent.
- In order to avoid that we have to shut down the service, this means that system will become unavailable.



CAP Theorem

- Thus, in case of a partitioned network, if we want to achieve consistency then availability cannot be achieved and if want to make our system available in spite of partition then consistency will be lost.



CAP in Practice

DB Type

Prioritizes

Examples

CP (Consistency + Partition)

Strong consistency, but may be unavailable during network issues

MongoDB, HBase

AP (Availability + Partition)

Always available, may return stale data

CouchDB, Cassandra

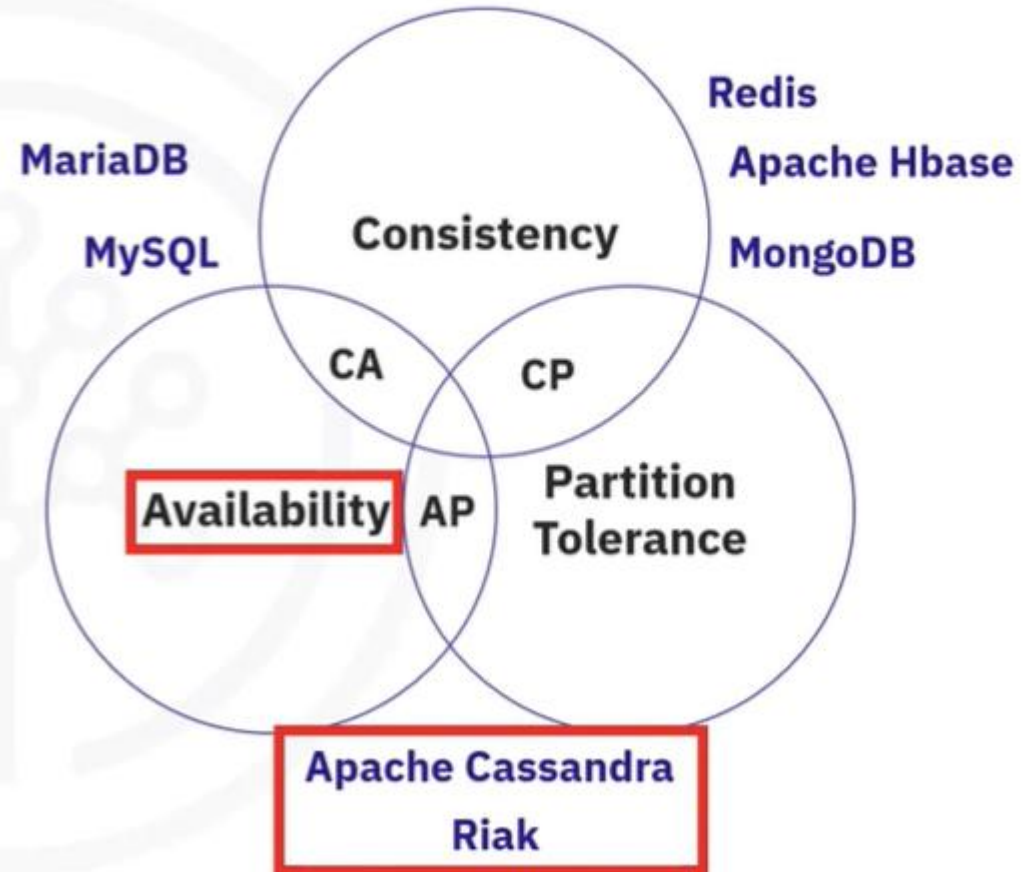
CA (Consistency + Availability)

Only works if there's no partition — impractical in real distributed systems

Traditional RDBMS (in non-distributed mode)

NoSQL: CP or AP

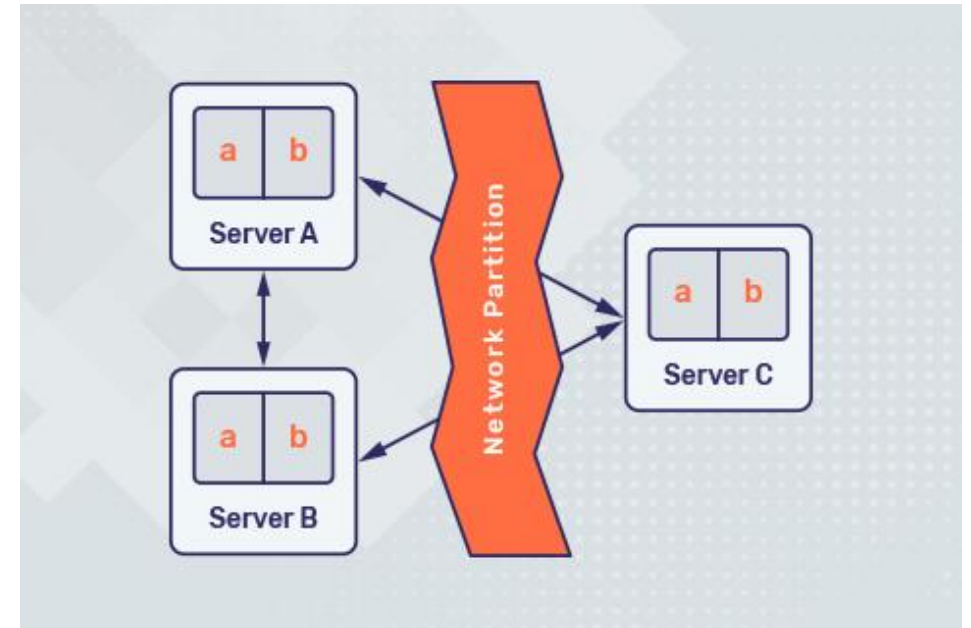
- NoSQL: A choice between consistency and availability
- MongoDB: Chooses consistency
- Apache Cassandra: Chooses availability
- These are tunable systems



CAP Theorem

E-Commerce Book Order

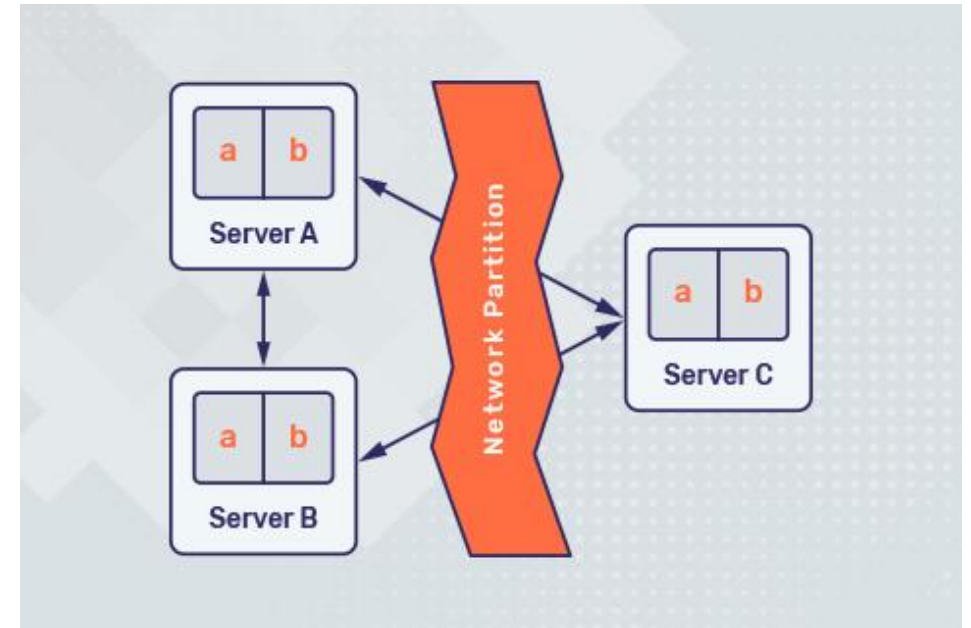
- There is a network partition and if we allow purchase of books in spite of network partitioning then we may receive the order of the book without having it in the stock.



CAP Theorem

E-Commerce Book Order

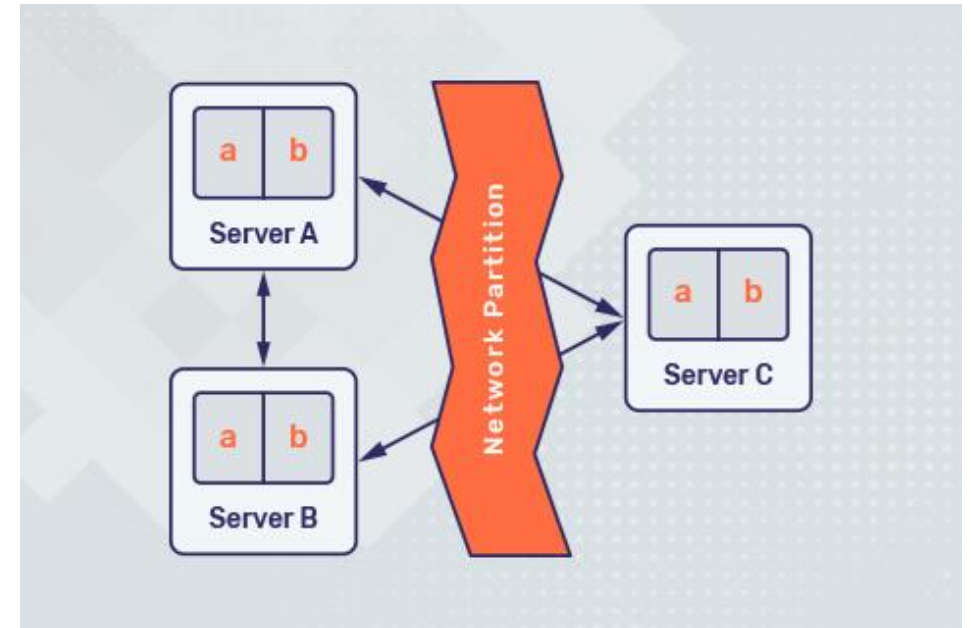
- In case of e-commerce website this is not a major issue, because for them order is important. They can still deliver the book by taking 2-3 more days or in worst case they can cancel the order. But we cannot do this for Railway Reservation System.



CAP Theorem

E-Commerce Book Order

- So, in case of E-commerce website NoSQL model is preferred for storing order as compared to RDBMS.
- But for payment RDBMS model will be preferred.



RDBMS to NoSQL: Overview

- NoSQL is not a de facto replacement for RDBMS
- RDBMS and NoSQL cater to different use cases
- RDBMS to NoSQL should be done based on careful case-by-case analysis based on performance and flexibility requirements

RDBMS or NoSQL

Consistency

Structured data
(Fixed Schema)

Transactions

Joins

RDBMS

OR

NoSQL

High Performance

Unstructured data
(flexible schema)

Availability

Easy Scalability

RDBMS to NoSQL: A mindset change

- **Data-driven** model to **query-driven** data model
 - RDBMS: Starts from the data integrity, relations between entities
 - NoSQL: Starts from your queries, not from your data models, based on the way the application interacts with the data
- **Normalized** to **Denormalized** data
 - NoSQL: Think how data can be structured based on your queries
 - RDBMS: Start from your normalized data and then build the queries

RDBMS to NoSQL: A mindset change

- **From ACID to BASE model**
 - Choose availability versus consistency
 - Apply the CAP Theorem: choose between availability and consistency
 - Consider availability, performance, geographical presence, high data volumes
- **NoSQL systems, by design, do not support transactions and joins except in limited cases**